CS683 — ADVANCED COMPUTER ARCHITECTURE

Programming Assignment 1: Task 1

Amey Dhoke 26M0795 | **Parthsinh Thakor** 26M0796 | **Deva Narayana B** 26M0797 | **Sumit Chavan** 26M0799

0.1 Experimental setup

Hardware setup

Component	Configuration
CPU	13th Gen Intel Core i5-13420H, topology is 4 P-cores + 4 E-cores
L1d cache	48 KiB per P-core
L2 cache	1.25 MiB per P-core
L3 cache	12 MiB
Memory	16 GiB
Vector ISA	SSE, AVX, AVX2, FMA — **no AVX-512**

Software setup

We isolated a P core using kernel boot parameters and then pinned the output binary to that core for taking measurements.

1 Task 1: 2D convolution

Baseline: `conv_naive`

Metric	Count
Instructions	4,377,401,359
Cycles	920,331,028
L1d cache loads	894,269,791
L1d cache load misses	4,812,940
MPKI	1.099

Table 1: Performance statistics of naive 2D convolution.

1.1 Task 1A: Loop unrolling and reordering

Q1:- In Conv Naive each element from kernel will be accessed H x W times and the small loop will be executed for large times adding up in the execution time, we can try to improve it by increasing the register reuse of kernel values and reordering to have longer stride. We can also further improve the bandwidth by performing more than 1 operation in 1 loop iteration.

Q2:- The reuse can be improved by loop reordering, we will have to take note of accumulator as when we reorder the loops, we will have to update the partial values that would have been added to accumulator variable(acc) in naive, directly on to the out matrix this will certainly increase our memory read write traffic but the duration of inner loops(2 hottest loops) will be more and thus we maximise the use of a particular kernel value. In summary we will access the kernel value once and reuse it H x W times rather than accessing it H x W times. Even though the each entry in input matrix will be accessed k x k times now compared to ones in the naive version here due to the stride the access pattern we again get benefit from spatial locality.

Q3:- In Loop Reordering the primary metric of evaluation will be speedup, as we compare the reordered code with naive code for reduction in execution in time. The next important metric to consider here is count of L1D misses as the code utilizes the spatial locality of cache, there will be noticeable difference in

this metric. The key changes in performance can be observed through the speedup and L1D misses . The metrics of instr_count and mpki will also help but not that much.
In Loop Unrolling the primary metric of evaluation will again be the speedup in execution time against the naive convolution program. The next metric which we can use is MPKI and instruction count as loop unrolling improves instruction level parallelism. L1D misses does not change much in this implementation as the access pattern remains the same but just multiple different values will be processed in a single iteration.

1.2 Task 1B: Tiling

The size of L1d cache in P-core is 48 KiB, and its size in E-Core is 32 KiB, we have used the performant cores through out our assignment.
We noticed that tile size 128 performs best for the graded case of 2048 x 2048 in terms of MPKI, it lagged a bit behind 32x32 in terms of speedup. We decided to give priority to MPKI because of errant nature of speedup metric.

Tile size ($H \times W$)	$N = 256$	$N = 512$	$N = 1024$	$N = 2048$
16 × 16	1.039	0.943	1.243	1.339
32 × 32	1.072	1.038	1.278	1.168
64 × 64	1.040	0.983	0.988	1.064
128 × 128	1.083	0.933	0.957	1.017

Table: L1d MPKI for different tile and matrix sizes

Tile size ($H \times W$)	$N = 256$	$N = 512$	$N = 1024$	$N = 2048$
16 × 16	1.13×	1.05×	0.98×	0.94×
32 × 32	1.11×	1.06×	0.99×	1.02×
64 × 64	1.26×	1.26×	1.02×	0.87×
128 × 128	1.09×	1.12×	1.14×	0.98×

Table: Speedup of tiled code over naive code at different tile and matrix sizes.

L1d MPKI vs matrix size, along with the naive baseline. The smaller tile sizes 16x16 and 32x32 start performing worse with increasing matrix sizes.
The Larger tile sizes remain better than baseline for the values observed.

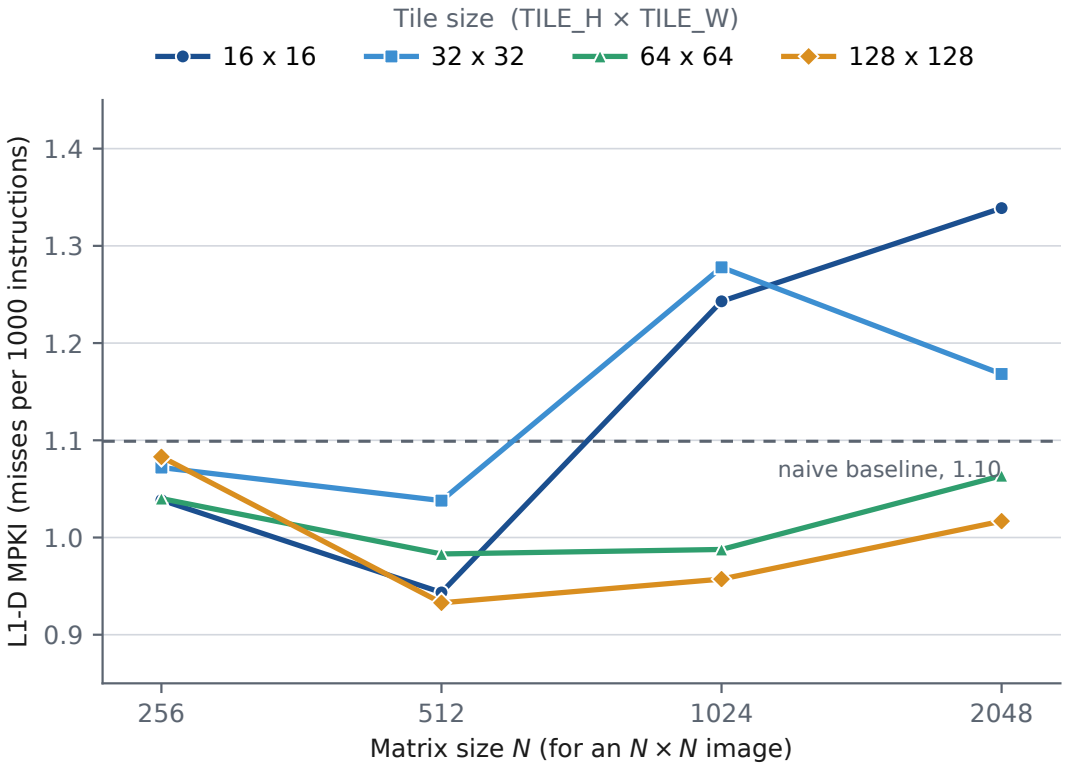

Figure 1: MPKI vs matrix size

Here there we noted that the baseline was varying very much causing the speedup to look very randomized, and no special trend being noticed.

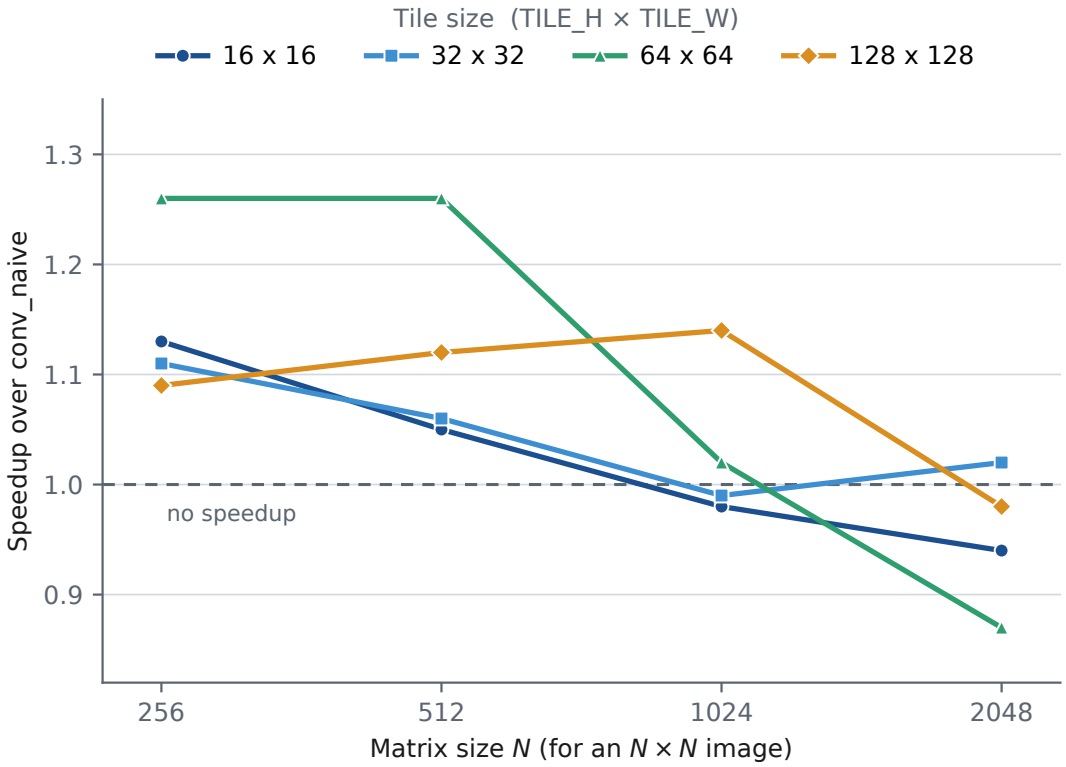

Figure 2: Speed up vs matrix size

Analysis deliverables

q1: When moving from naive to 2D convolution the MPKI barely improves, because we are working with a very small piece of memory to being with, thats why tiling which leverages the same solution does not do much better.

q2: The big tiles (64 × 64 and 128 × 128) keep MPKI low at every matrix size, while the small tiles (16 × 16 and 32 × 32) get clearly worse once the matrix is 1024 or larger, because a thin tile keeps coming back for the same rows and for big matrices those rows have already been thrown out of the 48 KiB L1 by then.

q3: Yes, but only a small one — 1.26× at $N = 256$ and 512 with 64 × 64 tiles, and at $N = 2048$ tiling is actually slower (0.87×), because the tiled loops run extra index and bound arithmetic and break the long straight reads that the hardware prefetcher is good at; tiling only the rows and mixing tiling with SIMD and unrolling should help.

1.3 Task 1C: SIMD

The number of Instructions in naive 2D convolution is 4,374,266,223

The number of Instructions in SIMD version for 2048 x 2048 matrix with Kernel of dimension 3 x 3 is 4,906,638,833

For this particular configuration of inputs we got a speedup of 7.93x

Deliverables

q1: for Naive code we record a total of 4.37 Billion instructions and when we shifted to SIMD implementation we recorded the total instruction count (Naive + SIMD) as 4.9 Billion instructions which shows that's SIMD implementation requires very less number of instructions as compared to naive code. This is possible because for a given operation we use Multiple data at once in SIMD implementation thus reducing the total number of instructions

q2: Yes, with SIMD Implementation we achieved speed up of around 7.71X. The speed up is possible because we are performing operation on multiple data (8 floats to be exact) at once which reduces the overall number of instructions.

q3: we used the default AVX2 intrinsic (m256) which includes the following functions:

`_mm256_loadu_ps()` : to load single point precision floats into a 256 width wide SIMD register.

`_mm256_fmadd_ps()` : this is used to Multiply two SIMD registers and add the result to another SIMD register all at once.

`_mm256_add_ps()` : this is used to add 2 SIMD registers.

`_mm256_storeu_ps()` : this is used to store the result of SIMD operations in another SIMD register.

Our general observation in comparing the 2 SIMD register widths we found that the 256-bit AVX2 beats 128-bit SSE at every matrix size and kernel - roughly 1.5-1.9× better, and it also maintains the speedup when the size of matrices increase.

Matrix size	K	Speedup	Instructions	L1d misses	MPKI
256	3	8.24×	79,908,283	133,260	1.668
256	5	9.59×	168,636,071	139,416	0.827
512	3	6.28×	309,934,501	393,169	1.269
512	5	9.02×	664,888,948	466,413	0.701
1024	3	7.52×	1,229,221,723	1,550,748	1.262
1024	5	9.16×	2,648,557,208	1,928,404	0.728
2048	3	7.93×	4,906,638,833	6,841,282	1.394
2048	5	9.10×	10,585,776,892	17,458,830	1.649

Table: For 256 Bit SIMD

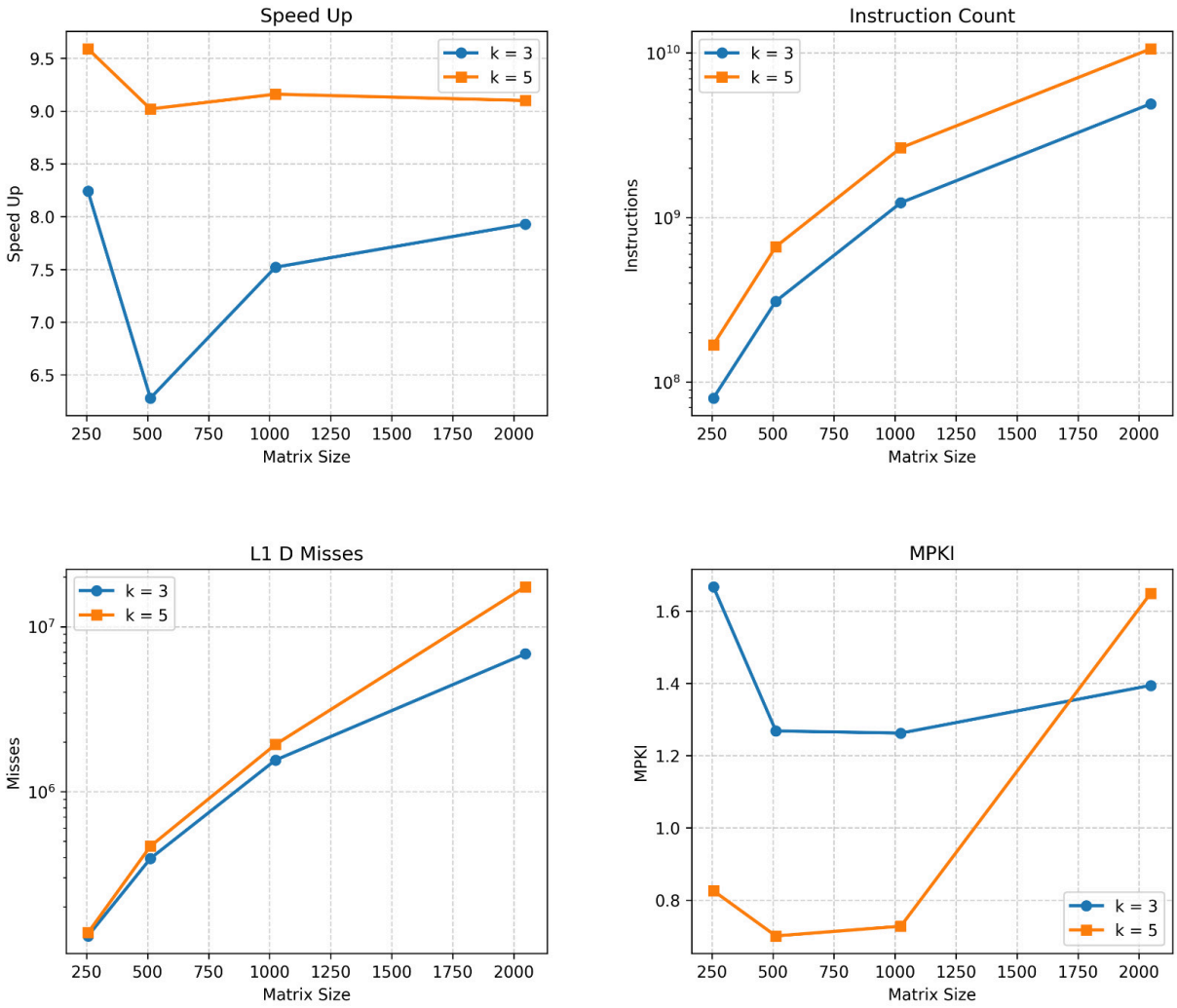

Figure 3: CPU metrics for 256bit SIMD

Matrix size	K	Speedup	Instructions	L1d misses	MPKI
256	3	5.48×	87,724,922	121,413	1.384
256	5	4.86×	185,825,985	132,613	0.714
512	3	5.62×	340,930,560	415,973	1.220
512	5	5.02×	733,174,587	468,340	0.639
1024	3	4.34×	1,354,016,801	1,640,039	1.211
1024	5	4.72×	2,922,516,815	1,958,168	0.670
2048	3	4.24×	5,403,258,689	7,360,450	1.362
2048	5	4.81×	11,679,133,866	16,389,588	1.403

Table: For 128 Bit SIMD

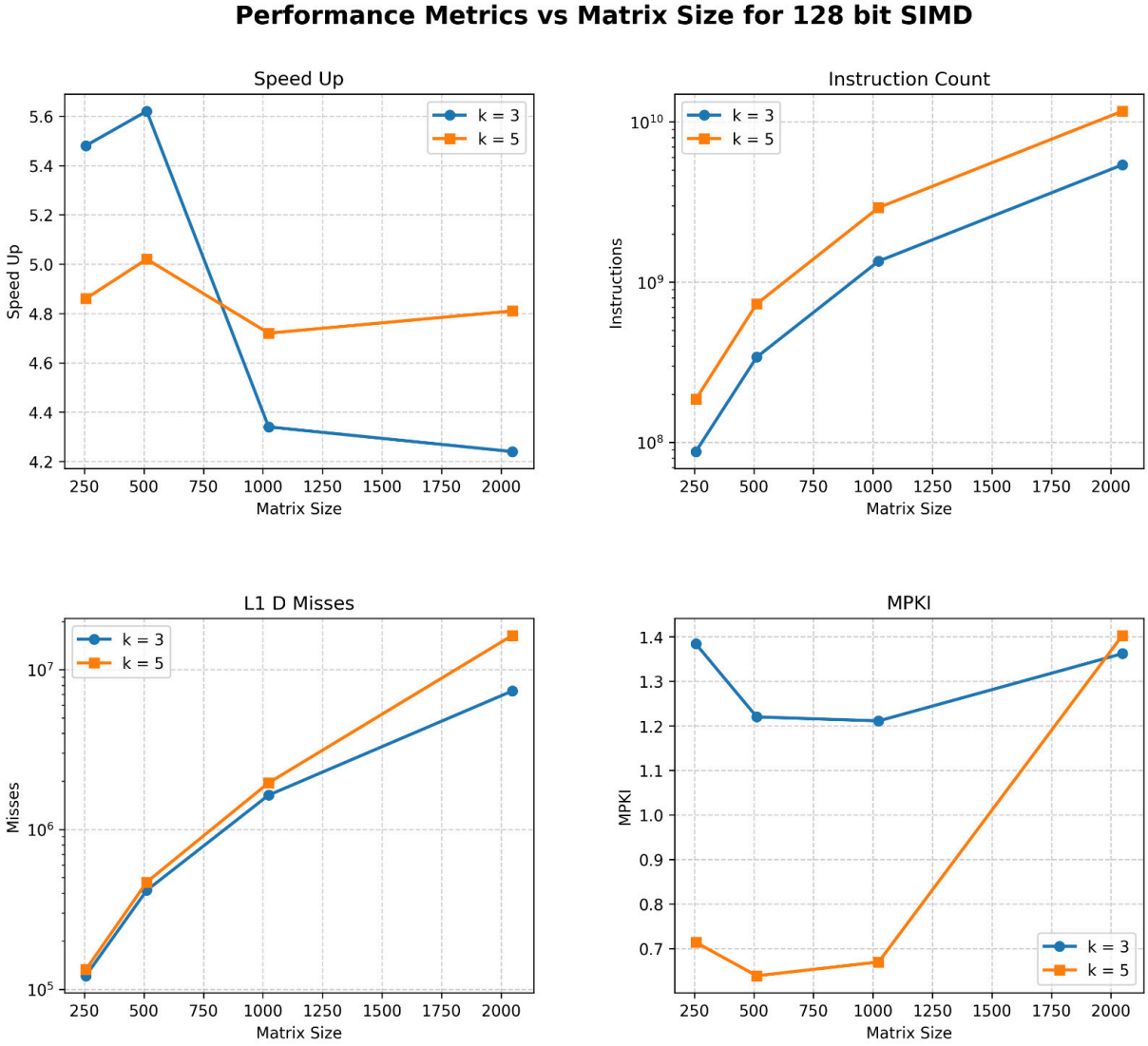

Figure 4: CPU metrics for 128bit SIMD

1.4 Task 1D: Sabka saath sabka vikaas

SIMD + unrolling is the best combination at every matrix size and both kernels , and adding tiling on top of it only makes things worse.

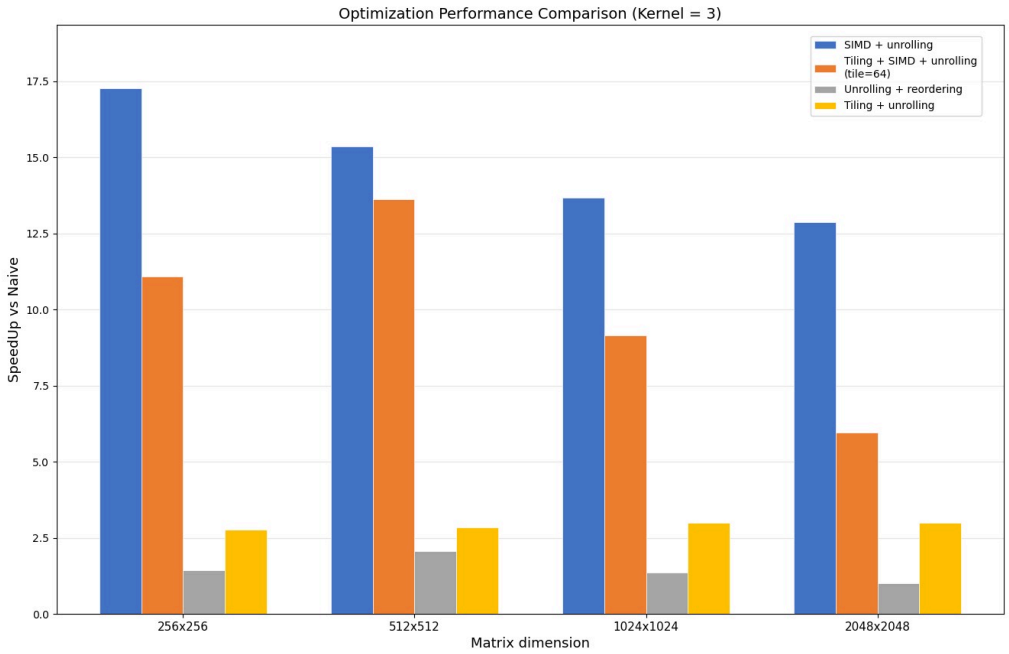

Figure 5: For $K = 3$ we compare different combinations of optimization techniques.

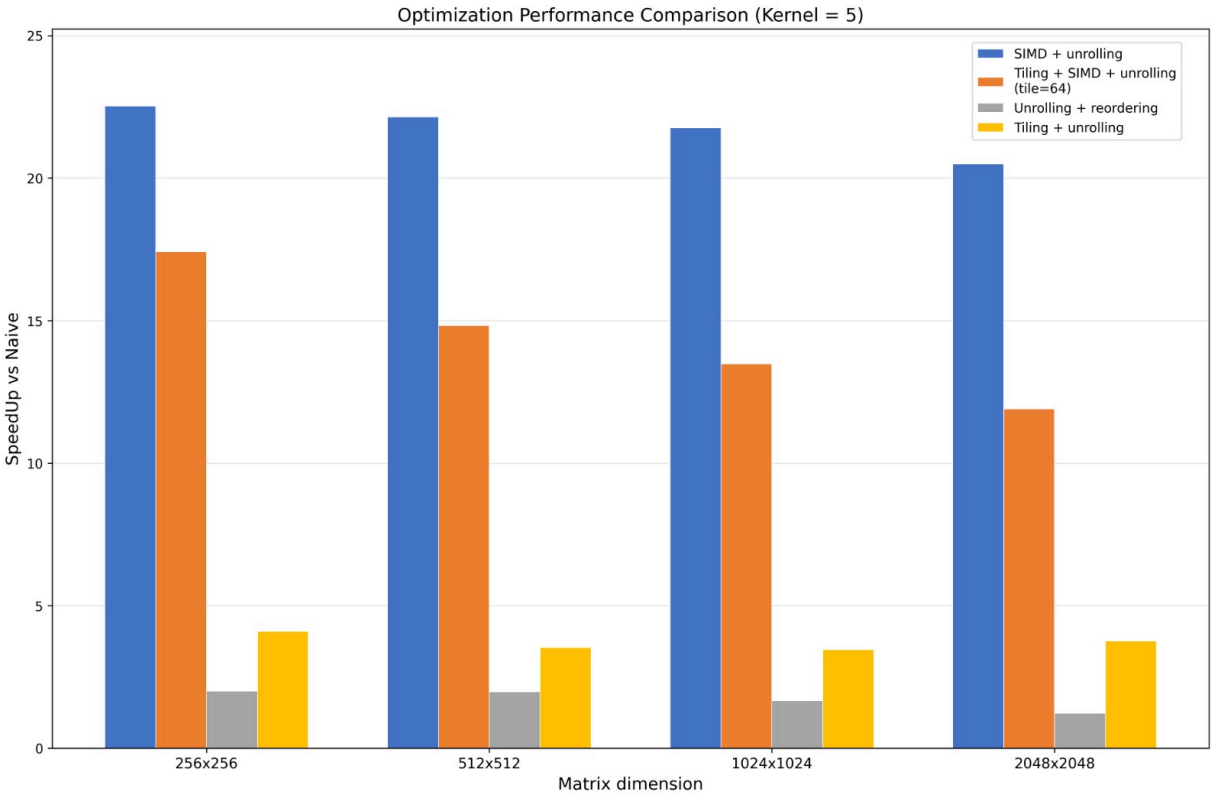

Figure 6: For $K = 5$ we compare different combinations of optimization techniques.